

TD 8 : Sockets client Serveur

2 Client Serveur TCP

On souhaite construire des classes de librairie C++ pour la gestion des sockets, rendant plus aisée leur utilisation dans une approche orientée objet.

Question 1. Construire un **operator** permettant d'afficher lisiblement un type **sockaddr_in**. On souhaite afficher l'**IP (v4)**, le **nom de l'hôte**, et le **numéro du port**. 显示IPv4、主机名和端口号

On rappelle les champs d'une adresse de socket du domaine internet :

- **sin_family** : discriminant de type d'adresse, prend la valeur **AF_INET**, 地址类型的判别值
- **sin_port** : le port au format internet,
- **sin_addr** : l'adresse IPv4; ce dernier champ n'est accessible que sur le type **sockaddr_in**, mais pas sur le type **sockaddr** mentionné dans la plupart de l'api, qui supporte d'autres domaines que internet. Cela induit des (cast) un peu maladroits mais nécessaires dans le code.

On utilisera **getnameinfo** pour obtenir le nom d'hôte et **inet_ntoa** pour afficher une IP. Attention à convertir vers le format hôte. Cf. slides 19 à 26 du cours 7.

名称解析可以通过域名系统 (DNS)、本地主机文件或其他命名机制来完成。这个函数可以被用来确定端口号对应的服务名称、进行IPv4或IPv6地址的反向DNS查询,或确定主机名对应的IP地址。getnameinfo 函数也可以用于将套接字结构中的IP地址或端口号转换为一个ANSI字符串。这个函数还可以被用于确定一个主机名对应的IP地址

Corrigé à la fin de Socket.cpp dans le corrigé de la question suivante.

The **getnameinfo** function is the ANSI version of a function that provides protocol-independent name resolution. The getnameinfo function is used to **translate the contents of a socket address structure to a node name** and/or a **service name**.

For IPv6 and IPv4 protocols, Name resolution can be by the **Domain-Name System (DNS)**, a local hosts file, or by other naming mechanisms. This function can be used to determine the host name for an IPv4 or IPv6 address, a reverse DNS lookup, or determine the service name for a port number. The getnameinfo function can also be used to convert an IP address or a port number in a sockaddr structure to an ANSI string. This function can also be used to determine the IP address for a host name.

Les autres c'est des bêtes fonctions de **conversions de format/string/host/network**. Celle là c'est spécial.

Question 2. Ecrire une classe Socket représentant une socket TCP. Elle porte en attribut un ledescriptor qui vaut -1 tant que la socket n'est pas connectée.

Socket.h

```
namespace pr {

class Socket {

    int fd; // 文件描述符, 未连接时为-1

public :

    Socket():fd(-1) {} // 默认构造函数, 将fd初始化为-1
    Socket(int fd):fd(fd) {} // 一个接受文件描述符的构造函数

    // tente de se connecter à l'hôte fourni
    void connect(const std::string & host, int port) ; // 连接到指定的host和port
    void connect(in_addr ipv4, int port);

    bool isOpen() const {return fd != -1;} // 判断套接字是否打开
    int getFD() { return fd ;} // 获取文件描述符

    void close(); // 关闭套接字

};

// 这个是一个操作符重载, 允许将套接字信息输出到输出流中
std::ostream & operator<< (std::ostream & os, struct sockaddr_in * addr);

}

#endif /* SRC_SOCKET_H_ */
```

Donc niveau résolution des noms, on a deux versions, une qui part d'un **string humain** et qui va vers un **format sockaddr_in** à l'aide **getaddrinfo**, une autre fonction qui utilise le **DNS** en background.

The getaddrinfo function is the ANSI version of a function that provides protocol-independent translation from **host name to address**. The getaddrinfo function returns results for the NS_DNS namespace. The getaddrinfo function aggregates all responses if more than one namespace provider returns information. For use with the IPv6 and IPv4 protocol, name resolution can be by the Domain Name System (DNS), a local hosts file, or by other naming mechanisms for the NS_DNS namespace.

Ca rend une chaine de résultats, d'où l'API un peu maladroite en C pour le nettoyer à la fin même si dans l'exemple on ne consulte que la première entrée.

Une fois qu'on a une ipV4, à part les cast pour l'API ça se passe assez directement. On remplit les champs et connect.

```
Socket.cpp

#include "Socket.h"
#include <unistd.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netdb.h>
#include <cstring>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <iostream>

namespace pr {

    // 关闭套接字
    void Socket::close() {
        if (fd != -1) { // 如果文件描述符不是-1, 即套接字是打开的
            shutdown(fd,1); // 关闭套接字的写功能
            ::close(fd); // 关闭文件描述符
        }
    }

    // 连接到指定的host和port
    void Socket::connect(const std::string & host, int port) {
        // but : resoudre l'adresse "humains" vers une adresse format machine
```

```
// 此函数用于将主机名解析为IP地址, 并尝试连接到指定的端口
struct addrinfo * addr; // 声明地址信息结构体 /* 填充目标结构体 */
/* Remplir la structure dest */ // 如果无法解析主机名到地址 (例如, DNS解析失败), 则打印错误并返回
// les choses à null ne servent pas ici
if (getaddrinfo(host.c_str(), /* service*/ nullptr, /* hints*/ nullptr, &addr) != 0) { // 解析地址
    perror("getaddrinfo");
    return; // 下面的代码处理addrinfo链表中的第一个地址, 这通常是我们需要的地址
} // 从addrinfo结构中提取socket地址, 并进行类型转换以获得IPv4地址
// champ "data" ai_addr (une adresse en reponse) du premier chainon de addrinfo
// dedans on trouve la chose utile : une adresse de socket, qu'il faut encore
// fixer vers un pointeur d'adresse de socket du domaine AF_INET
// on doit faire un (down)cast pour retyper sockaddr en sockaddr_in
in_addr ipv4 = ((struct sockaddr_in *) addr->ai_addr)->sin_addr;
// getaddrinfo alloue une liste, on desalloue poliment
freeaddrinfo(addr); // getaddrinfo函数会分配一个addrinfo结构体链表, 使用完毕后需要释放
connect (ipv4, port) ; // 使用提取的IPv4地址和端口号进行连接

}

// 定义Socket类中的connect函数, 这个函数接受一个IPv4地址和端口号
void Socket::connect(in_addr ipv4, int port) {
    // une adresse spécialisée pour internet
    sockaddr_in dest; // 声明目标地址的socket结构体
    dest.sin_family = AF_INET; // discriminant, aide à retrouver le type concret dans un switch case // 设置协议族为AF_INET, 即IPv4互联网协议
    dest.sin_addr = ipv4; // 将ipv4地址赋值给dest结构体的sin_addr部分
    dest.sin_port = htons(port); // host to network
    // 将主机字节序的port转换为网络字节序, 并赋值给dest结构体的sin_port部分
    // create socket // 创建一个socket, AF_INET指IPv4协议, SOCK_STREAM指定TCP类型的socket
    fd = socket(AF_INET,SOCK_STREAM, /* protocole TCP par default sur SOCK_STREAM*/ 0);
    if (fd <0) {
        perror("socket");
        return;
    }

    // ici on doit upcast de sockaddr_in (spécialisé) vers sockaddr plus général
    // en 00, pas de syntaxe, en C il faut un cast
    // on passe la taille réelle de l'objet sockaddr spécialisé
    if (::connect(fd,(struct sockaddr *) &dest, sizeof dest) <0) {
        perror("connect"); // 连接到目标地址, 如果连接失败, 打印错误
        ::close(fd); // 关闭socket并将文件描述符设置为-1后返回
        fd=-1;
        return;
    }
}

// sockaddr_in est API中广泛使用的结构, 例如在accept函数中用作发送方地址
// 目标是将机器格式的地址转换为人类可读的格式
// sockaddr_in est présent dans l'API a plein d'endroits, e.g. expéditeur dans un accept
// objectif : de l'adresse format machine vers du lisible humain
std::ostream & operator<< (std::ostream & os, struct sockaddr_in * addr) {
    char hname [1024]; // 用于存储主机名的字符串
    // obtient à partir de l'adresse machine le nom d'hôte
    if (getnameinfo((struct sockaddr *)addr,sizeof *addr, hname, 1024, nullptr, 0, 0) == 0) { // 尝试从socket地址中获取主机名
        os << " " << hname << " " << " ";
    }
    // inet_ntoa : produit la chain "127.0.0.1" à partir d'une adresse ipv4 sur 4 octets
    // inet_ntoa函数将网络地址转换为点分十进制字符串形式
    // ntohs : network to host // ntohs函数将网络字节顺序的端口号转换为主机字节顺序
    os << "inet_ntoa(addr->sin_addr) << " " << ntohs(addr->sin_port) << std::endl;
    return os;
}

重载了<<操作符, 以便可以直接使用std::cout或其他输出流对象来打印sockaddr_in结构的内容。代码首先尝试解析IP地址为主机名, 如果成功, 将其打印出来; 然后, 它将IP地址转换为点分十进制格式, 并将端口号从网络字节顺序转换为主机字节顺序, 最后将这些信息输出。
```

Question 3. Elaborer à présent une classe représentant une socket serveur.

ServerSocket.h

```
#ifndef SRC_SERVERSOCKET_H_
#define SRC_SERVERSOCKET_H_

#include "Socket.h"

namespace pr {

class ServerSocket {
    int sockfd; // 私有成员: 服务器套接字文件描述符

public:
    // Demarre l'ecoute sur le port donne
    ServerSocket(int port); // 构造函数: 在给端口上启动监听
    // 获取文件描述符
    int getFD() { return sockfd; }
    bool isOpen() const { return sockfd != -1; }
    // 接受一个连接请求, 返回与客户端通信的Socket对象
    Socket accept();
    // 关闭服务器套接字的公有成员函数
    void close();
};

} // ns pr
#endif /* SRC_SERVERSOCKET_H_ */
```

Dès la construction, on doit se mettre en attente de connexions TCP sur le port indiqué. L'appel à `accept` est bloquant, et rend une socket connectée au client.

On affichera l'adresse du client qui vient de se connecter à chaque connection (dans `accept`).

Donc la création d'un serveur TCP nécessite deux étapes de construction supplémentaires de la socket :

1. **bind** = déclarer la socket sur le réseau/lui donner une interface système avec la carte réseau qui est aussi un périphérique.

2. **listen** pour mettre en place une file d'attente de connection sur le port indiqué qui sera gérée par le système vu que l'on n'est pas toujours élu sur le CPU.

A ce stade, on a le droit de faire `accept` pour consommer un des clients dans cette file d'attente, s'il y en a (opération bloquante sinon). Ça rend une socket de connection bidirectionnelle où ce que l'on écrit est lu par le client est réciproquement en mode flux.

ServerSocket.h

```
#include "ServerSocket.h"
#include <cstring>
#include <unistd.h>
#include <iostream>

namespace pr {

// 创建TCP服务器套接字的两个额外步骤:
// bind - 将套接字绑定到网络接口, 这样操作系统就知道如何处理发往该套接字的数据。
// listen - 在指定的端口上设置连接请求的监听队列。
// 完成后, 服务器可以使用accept函数来接受客户端的连接请求, 该函数是
// 阻塞的, 并且返回一个新的Socket对象, 用于与客户端进行通信。

// ServerSocket类的构造函数实现
ServerSocket::ServerSocket(int port) : sockfd(-1) {

    // create socket
    int fd = socket(AF_INET, SOCK_STREAM, 0);
    if (fd == -1) {
        perror("create socket");
        return;
    }

    // bind // 绑定套接字到指定端口 // 定义套接字地址结构体
    struct sockaddr_in sin; // Nom de la socket de connexion */
    // 初始化地址结构体
    memset(&sin, 0, sizeof(sin)); // utile ?
    sin.sin_family = AF_INET; // 设置地址族为IPv4
    sin.sin_addr.s_addr = htonl(INADDR_ANY); // on attend sur n'importe quelle
    // interface de la machine // 套接字接受任何发往本机IP的连接
    sin.sin_port = htons(port); // host to network
    // 将端口号从主机字节序转换为网络字节序

    /* nommage, meme probleme de typage sockaddr que dans la Socket */
    if (bind(fd, (struct sockaddr *) &sin, sizeof(sin)) < 0) {
        perror("bind"); // 绑定地址结构体到套接字文件描述符
        // on veut le close de unistd, pas le close membre de cette classe
        ::close(fd);
        return;
    }

    // listen // 监听端口, 设置最大同时连接请求数为50
    if (listen(fd, 50) < 0) {
        perror("listen");
        ::close(fd);
        return;
    }

    // 如果上述步骤都成功了, 将文件描述符赋值给类成员sockfd
    // success !
    sockfd = fd;
}

// accept函数实现: 接受连接请求, 返回一个Socket对象
Socket ServerSocket::accept() {
    struct sockaddr_in exp; // 定义对方地址结构体
    socklen_t len = sizeof(exp); // 定义地址长度
    // on qualifie sinon le compilateur // 等待并接受连接请求
    int scom = ::accept(sockfd, (struct sockaddr *) &exp, &len);
    if (scom < 0) {
        perror("accept");
    } else {
        // en appui sur operator << développé plus haut
        std::cout << "Accepted connection from " << &exp << std::endl;
    }
    return scom; // 返回通信的套接字文件描述符
}

void ServerSocket::close() {
    if (sockfd != -1) {
        ::close(sockfd); // 关闭套接字
    }
}

}
```

```
ServerSocket::ServerSocket(int port) : sockfd(-1) {

    // create socket
    int fd = socket(AF_INET, SOCK_STREAM, 0);
    if (fd == -1) {
        perror("create socket");
        return;
    }

    // bind // 绑定套接字到指定端口 // 定义套接字地址结构体
    struct sockaddr_in sin; // Nom de la socket de connexion */
    // 初始化地址结构体
    memset(&sin, 0, sizeof(sin)); // utile ?
    sin.sin_family = AF_INET; // 设置地址族为IPv4
    sin.sin_addr.s_addr = htonl(INADDR_ANY); // on attend sur n'importe quelle
    // interface de la machine // 套接字接受任何发往本机IP的连接
    sin.sin_port = htons(port); // host to network
    // 将端口号从主机字节序转换为网络字节序

    /* nommage, meme probleme de typage sockaddr que dans la Socket */
    if (bind(fd, (struct sockaddr *) &sin, sizeof(sin)) < 0) {
        perror("bind"); // 绑定地址结构体到套接字文件描述符
        // on veut le close de unistd, pas le close membre de cette classe
        ::close(fd);
        return;
    }

    // listen // 监听端口, 设置最大同时连接请求数为50
    if (listen(fd, 50) < 0) {
        perror("listen");
        ::close(fd);
        return;
    }

    // 如果上述步骤都成功了, 将文件描述符赋值给类成员sockfd
    // success !
    sockfd = fd;
}

// accept函数实现: 接受连接请求, 返回一个Socket对象
Socket ServerSocket::accept() {
    struct sockaddr_in exp; // 定义对方地址结构体
    socklen_t len = sizeof(exp); // 定义地址长度
    // on qualifie sinon le compilateur // 等待并接受连接请求
    int scom = ::accept(sockfd, (struct sockaddr *) &exp, &len);
    if (scom < 0) {
        perror("accept");
    } else {
        // en appui sur operator << développé plus haut
        std::cout << "Accepted connection from " << &exp << std::endl;
    }
    return scom; // 返回通信的套接字文件描述符
}

void ServerSocket::close() {
    if (sockfd != -1) {
        ::close(sockfd); // 关闭套接字
    }
}

}
```

Question 4. Ecrivez un code client, qui se connecte à un serveur donné, lui envoie une donnée (un entier), puis lis sa réponse (un entier) avant de quitter.

Des versions de plus en plus riche dans le corrigé. A la fin on fait une boucle de dix émissions/réponse. Le serveur correspondant sort quand on lui envoie 0.

```

client.cpp

#include "ServerSocket.h"
#include <iostream>
#include <unistd.h>
#include <string>

// 基础版, 只进行一次数据发送和接收
int main00() {
    pr::Socket sock;
    sock.connect("localhost", 1664); // 连接到本地主机的1664端口
    int N=42;
    write(sock.getFD(), &N, sizeof(int)); // 向套接字写入整数N
    read(sock.getFD(), &N, sizeof(int)); // 从套接字读取整数到N
    std::cout << N << std::endl;
    return 0;
}

// avec controle
int main() { // 加入了错误检查

    pr::Socket sock;
    // 连接到本地主机的1664端口
    sock.connect("localhost", 1664);

    if (sock.isOpen()) { // 如果套接字成功打开
        int fd = sock.getFD();
        int i = 10;
        ssize_t msz = sizeof(int);
        if (write(fd, &i, msz) < msz) { // 写入整数到套接字
            perror("write");
        }
        std::cout << "envoyé =" << i << std::endl;
        int lu;
        auto nblu = read(fd, &lu, msz); // 读取整数到lu
        if (nblu == 0) { // 如果没有读取到数据, 输出服务器连接结束信息
            std::cout << "Fin connexion par serveur" << std::endl;
        } else if (nblu < msz) {
            perror("read");
        }
        std::cout << "lu =" << lu << std::endl;
    }

    return 0;
}

// 在接收到数据为0时退出循环, 这通常是
// 服务器端告诉客户端停止通信的信号
// avec une boucle, on attend un 0
int main() { // 带有循环的main函数版本, 当读取到0时退出循环

    pr::Socket sock;

    sock.connect("localhost", 1664);

    if (sock.isOpen()) { // 如果套接字成功打开
        int fd = sock.getFD();

        ssize_t msz = sizeof(int);
        for (int i = 10; i >= 0; i--) { // 从10递减至0
            if (write(fd, &i, msz) < msz) {
                perror("write"); // 写入整数到套接字本
                break;
            }
            std::cout << "envoyé =" << i << std::endl;

            int lu;
            auto nblu = read(fd, &lu, msz);
            if (nblu == 0) { // 如果没有读取到数据, 输出服务器连接结束信息
                std::cout << "Fin connexion par serveur" << std::endl;
                break;
            } else if (nblu < msz) {
                perror("read");
                break;
            }
            std::cout << "lu =" << lu << std::endl;
        }
    }

    return 0;
}

```

Question 6. On souhaite généraliser le code du serveur, écrire une classe TCPServer qui encapsule le comportement du serveur. La classe porte une opération bool startServer(int port) qui démarre l'écoute sur le port ciblé à l'aide d'une ServerSocket. A la construction on lui passe un ConnectionHandler : un objet devant gérer une session avec un client donné.

Question 5. Ecrivez un code serveur, qui attend des connexions, et quand un client se connecte commence par lire un message (un entier) puis renvoie un message (un entier), puis se remet en attente de connexion.

创建一个服务器套接字, 不断地等待客户端连接。每当有客户端连接时, 它读取客户端发送的一个整数消息, 然后对这个整数进行加一操作, 再将结果发送回客户端。

Des versions de plus en plus riche dans le corrigé. A la fin on fait une boucle de dix émissions/réponse. Le serveur correspondant sort quand on lui envoie 0.

```

server.cpp

#include "ServerSocket.h"
#include <iostream>
#include <unistd.h>

int main00() { // 在1664端口上创建服务器套接字
    pr::ServerSocket ss(1664);
    // 无限循环, 持续等待客户端连接
    while (1) { // 接受客户端连接, 返回一个套接字
        pr::Socket sc = ss.accept();
        // 获取与客户端通信的文件描述符
        int fd = sc.getFD();

        int lu;
        read(fd, &lu, sizeof(int));
        std::cout << "lu =" << lu << std::endl;
        lu++;
        write(fd, &lu, sizeof(int));
        sc.close();
    } // 关闭服务器套接字
    ss.close();

    return 0;
}

int main() { // 创建一个在端口1664上监听的服务器套接字
    pr::ServerSocket ss(1664);
    // 无限循环, 等待客户端连接
    while (1) { // 接受一个客户端连接, 创建一个通信套接字
        pr::Socket sc = ss.accept();
        // 获取通信套接字的文件描述符
        int fd = sc.getFD();

        // 设置读取大小为一个整数的大小
        ssize_t msz = sizeof(int);
        while (1) { // 再次进入无限循环, 用于与客户端通信
            int lu; // 用于存储从客户端读取的整数
            auto nblu = read(fd, &lu, msz);
            if (nblu == 0) { // 如果读取字节数为0, 表示客户端关闭了连接
                std::cout << "Fin connexion par client" << std::endl;
                break;
            } else if (nblu < msz) {
                perror("read");
                break;
            }
            std::cout << "lu =" << lu << std::endl;
            // 如果读取的整数为0, 结束通信
            if (lu == 0) {
                break;
            }
            lu++; // 将修改后的整数写回客户端
            if (write(fd, &lu, msz) < msz) {
                perror("write");
                break;
            }
            std::cout << "envoyé =" << lu << std::endl;
        }
        sc.close();
    }

    ss.close();
    return 0;
}

```



```
#ifndef SRC_CONNECTIONHANDLER_H_
#define SRC_CONNECTIONHANDLER_H_

#include "Socket.h"

namespace pr {

// une interface pour gerer la communication
class ConnectionHandler {
public:
    virtual void handleConnection(Socket s) = 0;
    // une copie identique
    virtual ConnectionHandler * clone() const = 0;
    // pour virtual
    virtual ~ConnectionHandler() {}
};
#endif /* SRC_CONNECTIONHANDLER_H_ */
```

处理与客户端的通信，例如读取数据、处理请求并发送响应。参数Socket s是与客户端通信的套接字。

// gerer une conversation sur une socket

virtual void handleConnection(Socket s) = 0;

// une copie identique

virtual ConnectionHandler * clone() const = 0;

// pour virtual

virtual ~ConnectionHandler() {}

使得服务器能够为每个新的客户端连接创建一个ConnectionHandler的新实例，这是实现原型设计模式的一部分。原型设计模式允许对象能够复制自身，以便创建新对象时不需要知道对象属于哪个具体类。

Il est possible que chaque session avec un client ait un état interne, e.g. le client s'est authentifié, d'où la nécessité de cloner le handler à chaque nouvelle connexion d'un client. Cette approche correspond au design pattern *Prototype*.与客户端的每个会话都可能具有内部状态，例如客户端已经过身份验证，因此需要在客户端的每个新连接时克隆处理程序。这种方法对应于原型图案设计。

TCPServer.h

```
#ifndef SRC_TCPSERVER_H_
#define SRC_TCPSERVER_H_

#include <thread>
#include "ServerSocket.h"
#include "ConnectionHandler.h"

namespace pr {

// un serveur TCP, la gestion des connections est déléguée
class TCPServer { // TCP服务器类，负责启动和管理服务器，以及处理客户端连接 // 用于监听客户端连接的线程指针
    std::thread * waitT; // le thread d'attente de connexion s'il est instancie
    ServerSocket * ss; // la socket d'attente si elle est instanciee
    ConnectionHandler * handler; // le gestionnaire de session passe a la cstru
    int killpipe; // introduit a la fin, pour traiter le stopServer

public:
    // 构造函数，初始化成员变量，接受一个处理器以处理连接
    TCPServer(ConnectionHandler * handler): waitT(nullptr),ss(nullptr),handler(handler
    ),killpipe(-1) {}
    // Tente de creer une socket d'attente sur le port donné
    // engendre un thread d'attente de connections
    // des variantes au fil des questions // 启动服务器，监听指定端口
    bool startServer0 (int port); // 基础启动方法
    bool startServer1 (int port); // 使用线程异步处理连接的启动方法
    bool startServer2 (int port); // 支持通过信号停止服务器的异步启动方法

    // ferme la socket
    // ce qui interrompt le thread d'attente de connection
    void stopServer () ; // 停止服务器，关闭监听线程和套接字
};

} // ns pr

#endif /* SRC_TCPSERVER_H_ */
```

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32

Question 8. Comment gérer l'opération symétrique **stopServer**, qui doit **interrompre le thread** d'attente et **fermer proprement la socket** ?

中断等待连接的线程并且适当地关闭套接字

Une vraie question plus subtile qu'il n'y parait. Le problème : le thread qu'on souhaite arrêter est bloqué sur le accept.

On n'a pas de variante de accept avec un timeout ou autre.

Option 0 : provoquer un **EINTR** sur **accept**, i.e. interrompre le thread. Ca signifie utiliser un **signal**. MAIS, on est en **multi-thread**. Du coup **pas de garantie** (du tout) que le signal soit délivré à un thread en particulier au sein du processus. 不能保证信号一定会被目标线程接收

Option 1 : utiliser une **multi-attente** avec une variante de **select/poll**. select permet d'attendre plusieurs descripteurs avec en plus un **timeout**. Quand une socket d'attente de connection (état listen) reçoit une demande cela déclenche un état "read".

On peut donc **boucler sur un select avec un timeout**, et périodiquement consulter une variable membre de Server qui aura été mise à 0 pour signifier qu'on en a marre. Pas génial, attente semi active tout ça.

Option 2 : on ajoute un **autre filedescriptor** à la liste passée à **select**, crée pour l'occasion. Par exemple un **petit pipe**, quand on écrit dedans (n'importe quoi) ça veut dire qu'il faut **quitter**. C'est cette option qui est fait dans le corrigé.

在 select 调用中添加另一个文件描述符，专门为此创建，比如一个小的管道。向这个管道写入数据（任何数据）表示需要退出

```
namespace pr {
    // 基本版本: 同步客户端通信
    // Version basique : mobilise le client ( Question 6 )
    bool TCPServer::startServer0(int port) {
        ss = new ServerSocket(port); // 在指定端口上创建一个新的服务器套接字
        if (ss->isOpen()) { // 如果服务器套接字成功打开
            while (1) { // 无限循环, 等待客户端连接
                std::cout << "En attente sur accept" << std::endl;
                auto sc = ss->accept(); // 接受一个客户端连接
                if (sc.isOpen()) { // 如果客户端套接字成功打开
                    auto copy = handler->clone(); // 克隆一个连接处理器
                    copy->handleConnection(sc); // 使用克隆的处理器处理连接
                    delete copy;
                }
            }
            // inaccessible
            return true;
        } // 如果服务器套接字没有成功打开, 则返回false
        return false;
    }

    // 异步版本: 允许服务器在不阻塞的情况下启动 (
    // version asynchrone permet de demarrer le serveur sans bloquer (Question 7)
    bool TCPServer::startServer1(int port) {
        if (waitT == nullptr) { // 如果当前没有等待线程
            ss = new ServerSocket(port); // 在指定端口上创建一个新的服务器套接字
            if (ss->isOpen()) { // 创建一个新的线程来处理客户端连接
                waitT = new std::thread([](TCPServer * serv){
                    while (1) {
                        std::cout << "En attente sur accept" << std::endl;
                        auto sc = serv->ss->accept();
                        std::cout << "Après accept" << std::endl;
                        if (sc.isOpen()) { // 克隆一个连接处理器
                            auto copy = serv->handler->clone();
                            copy->handleConnection(sc); // 使用克隆的处理器处理连接
                            delete copy;
                        }
                    }
                }, this);
                return true; // 返回true表示服务器成功启动
            }
        }
        // 如果服务器套接字没有成功打开或已有等待线程, 则返回false
        return false;
    }

    // 异步版本: 通过管道可以请求服务器停止
    // version asynchrone, mais on peut kill le serveur, fait avec un pipe (Question 8)
    bool TCPServer::startServer2(int port) {
        if (waitT == nullptr) { // 如果当前没有等待线程
            ss = new ServerSocket(port);
            if (ss->isOpen()) {
                int pipefd[2]; // 创建一个管道文件描述符数组
                if (pipe(pipefd) < 0) { // 创建管道
                    perror("pipe");
                    return false;
                }
                // 创建一个新的线程来处理客户端连接
                waitT = new std::thread([](TCPServer * serv, int readpipe){
                    while (1) {
                        // Construire un set pour le select (attente simultanee)
                        fd_set read; // 为select构建一个集合 (用于同时等待)
```

```
        // 确定三个集合 (读集合、写集合和异常集合) 中最大的文件描述符编号, 并加一。
        // select 函数会监控所有编号小于这个值的文件描述符。在许多实现中, 您需要传递最大文件描述符值加一给系统, 以便系统检查所有的文件描述符, 直到这个最大值
        FD_ZERO(&read); // 清空文件描述符集合
        FD_SET(readpipe, &read); // 将读管道加入文件描述符集合
        FD_SET(serv->ss->getFD(), &read); // 将服务器套接字的文件描述符加入集合
        // premier argument = plus grand fildescriptor dans un des set + 1 第一个参数 = 文件描述符集合中最大的文件描述符 + 1
        // on peut comprendre que fd_set est un genre de bitset, on donne sa taille ici. // 我们可以理解fd_set是一种位集, 这里指定其大小
        select(std::max(readpipe, serv->ss->getFD()) + 1, &read, /* write=nullptr, /*except=nullptr, /*timeout=nullptr */);
        // 使用select函数来等待文件描述符集合中任意文件描述符的状态改变
        // au retour du select, read est modifié pour contenir le ou les fd qui nous ont reveillé
        if (FD_ISSET(readpipe,&read)) { // 如果读管道的文件描述符状态改变
            std::cout << "asked to quit" << std::endl;
            return;
        }
        // sinon ben c'est la socket qui nous a réveillé : un client s'est pointé
        auto sc = serv->ss->accept();
        if (sc.isOpen()) {
            auto copy = serv->handler->clone();
            copy->handleConnection(sc);
            // En version Pool de thread
            // pool.submit(new JobHandler(copy,sc));
            // et on enlève le delete (fait par JobHandler)
            delete copy;
        }
        // pipefd[0] 传递给线程, 如果线程需要从管道中读取数据, 它将使用这个文件描述符
        }, this, pipefd[0]);
        killpipe = pipefd[1]; // 保存写入端的文件描述符, 用于发送停止服务器的信号
        return true;
    }

    void TCPServer::stopServer() {
        if (waitT != nullptr) { // 如果等待线程存在
            int n = 1; // 向管道写入数据以通知停止服务器
            write(killpipe,&n,sizeof(int));
            waitT->join(); // 等待线程结束
            delete waitT;
            delete ss;
            close(killpipe);
            killpipe = -1; // 将管道文件描述符设置为无效
            waitT = nullptr; // 将等待线程指针设置为nullptr
        }
    }
}
```

Question 7. On souhaite que l'invocation à `startServer` ne bloque pas l'appelant, mais crée au contraire un nouveau thread qui s'occupe d'attendre des connexions.

调用 `startServer` 时不会阻塞调用者, 相反, 它会创建一个新的线程来等待连接。

Cf version 1 de `startServer`. 为每个连接启动一个新线程, 至少传递给它必须克隆的处理器 (handler) 和等待连接的套接字。在这里, 我直接传递了整个 `Server`。En gros on lance un nouveau thread pour chaque connection, on lui passe au minimum : le handler qu'il doit clone, la socket d'attente de connection. Ici je lui passe brutalement le `Server` entier.

Question 9. Proposer un **adapter** pour permettre l'**encapsulation** de la gestion d'une session de discussion avec un client dans un **Job** muni d'une unique méthode **void run()** que l'on peut soumettre à un Pool de thread.

适配器，以便在一个线程池（Pool de thread）中管理与客户端的对话
会话，该会话被封装在一个包含单一 void run() 方法的 Job 类中

```
#ifndef SRC_JOB_H_
#define SRC_JOB_H_

class Job {
public:
    virtual void run () = 0;
    virtual ~Job() {};
};
#endif /* SRC_JOB_H_ */
```

JobHandler.h

```
#ifndef SRC_JOBHANDLER_H_
#define SRC_JOBHANDLER_H_

#include "Job.h"
#include "ConnectionHandler.h"

namespace pr {
    // 定义一个名为 JobHandler 的类，继承自 Job 类
    class JobHandler : public Job {
        ConnectionHandler * ch; // 定义一个指向 ConnectionHandler 的指针
        Socket s; // 定义一个 Socket 对象
    public:
        // 构造函数，初始化 ch 和 s
        JobHandler(ConnectionHandler * ch, Socket s):ch(ch),s(s) {}
        void run() { ch->handleConnection(s); delete ch ;}
    };
    // 实现 run 方法，处理连接然后删除 ConnectionHandler 对象
}

#endif /* SRC_JOBHANDLER_H_ */
```